

Mei console specification v0.1

Sep 30, 2026 · @Jack Galler

Overview

Mei is a 3D fantasy console modelled on PlayStation-era hardware: a small RISC CPU, a vector unit for geometry, and a GPU that only fills 2D triangles. "Mei" (明, "bright", written as sun plus moon) is a working name. The Sun/Moon theme is branding only and has no effect on the hardware.

Design goals:

- **The look comes from the hardware rules.** Affine textures, pixel-snapped vertices, sort-based depth and dither are how the machine works, not post-effects.
- **Limits are real.** A cycle budget, a triangle limit and fixed memory sizes are enforced by the machine.
- **Deterministic.** All math is fixed-point and input is sampled once per frame, so a cart runs identically everywhere.
- **Runs in a browser tab.** The emulator is C compiled to WebAssembly.

Area	Spec
Display	320×240 at 60 fps, 15-bit colour, 4×4 ordered dither
CPU	32-bit RISC, 59 instructions, 500,000 cycles per frame
Vector unit	8 registers, each four 16.16 fixed-point lanes
RAM	2 MB
Cart ROM	2 MB
VRAM	1 MB: two framebuffers, 4,096 palette colours, 16 texture slots
GPU	2D triangles and quads, 2,000 triangles per frame, four blend modes
Audio	8 channels, 22,050 Hz, 8- or 16-bit samples

Area	Spec
Input	Two controllers, each with d-pad, one analog stick, A B X Y, L R, start
Language	Custom, statically typed, compiled to the CPU's machine code
Implementation	C, compiled to WebAssembly

Video

The GPU is a software rasterizer that draws 2D triangles into a 320×240 framebuffer; it has no notion of depth. Everything three-dimensional happens on the CPU and vector unit before a triangle is submitted.

Framebuffer. Each pixel is 16 bits holding 15-bit colour: bits 0–4 red, 5–9 green, 10–14 blue, bit 15 unused. There are two framebuffers. Carts draw into the back buffer, and the buffers swap automatically at `vsync`.

Rule	Behaviour	Visible result
Texture mapping	Affine (linear in screen space)	Textures warp on large or steep polygons
Texture sampling	Nearest texel, no mipmaps	Crisp, chunky texels
Vertex positions	Whole pixels only	Vertices wobble as objects move
Depth	None; later packets draw over earlier ones	Sorting glitches between nearby polygons
Colour math	8 bits per channel, reduced to 5 on output	Banding, hidden by dither
Dither	4×4 ordered pattern, switchable	The crosshatch look
Clipping	To the screen edges only	Polygons crossing behind the camera must be handled by the cart

Dither. When bit 0 of `GPU_CTRL` is set, the GPU adds an offset from a 4×4 matrix to each 8-bit channel, clamps, then drops the low 3 bits. The offset is chosen by the pixel's x and y modulo 4. The proposed matrix is the one the PlayStation used:

	x=0	x=1	x=2	x=3
y=0	-4	0	-3	1
y=1	2	-2	3	-1
y=2	-3	1	-4	0
y=3	3	-1	2	-2

Texture tinting. A texel is multiplied by the polygon colour, where 128 means unchanged. Values above 128 brighten, up to double, and results clamp at 255.

Transparency. Palette index 0 is never drawn. Semi-transparent packets blend with the framebuffer using one of four modes (see GPU packets).

Budget. The GPU draws at most 2,000 triangles per frame. Extra triangles are dropped and a status bit is set. Drawing costs no CPU cycles.

CPU

The CPU is a 32-bit RISC core with fixed 32-bit instructions, little-endian, with no floating point, no flags register, no delay slots and no interrupts. Branches compare two registers and jump in one instruction.

Scalar registers. There are 16, `r0` – `r15`, each 32 bits. `r0` always reads zero and ignores writes.

Vector registers. There are 8, `v0` – `v7`, each holding four 32-bit lanes named x, y, z, w. Lanes normally hold 16.16 fixed-point values. All eight are general-purpose; the transform instruction reads its matrix rows from `v4` – `v7`.

Fixed-point. A 16.16 value is a signed 32-bit integer where 65,536 represents 1.0. `fmul` and `fdiv` multiply and divide these with a 64-bit intermediate.

Reset state.

Register	Value at reset
pc	0x200000 (start of cart ROM)
r14	0x200000 (top of RAM; the stack grows down)
Everything else	0

Calling convention. This is used by the compiler and not enforced by the hardware.

Registers	Role
r1 – r4	Arguments; r1 also holds the return value
r5 – r8	Temporaries, not preserved across calls
r9 – r13	Preserved across calls
r14	Stack pointer
r15	Return address, set by call and callr
v0 – v7	Not preserved across calls

Instruction encoding

Every instruction is one 32-bit word with a 6-bit opcode in the top bits, so a single switch on bits 31–26 decodes everything. There are four formats.

Format	Bits 31–26	Bits 25–22	Bits 21–18	Bits 17–14	Bits 13–0
R	opcode	a	b	c	must be zero
I	opcode	a	b	imm18 (bits 17–0)	
U	opcode	a	imm22 (bits 21–0)		
J	opcode	imm26 (bits 25–0)			

- **Register fields** are 4 bits. Vector registers use the same fields, and values 8–15 fault.
- **Sign extension.** addi, slti, memory offsets and branch offsets sign-extend imm18. andi, ori and xori zero-extend it. Shifts use its low 5 bits.
- **Branch targets** are $pc + 4 + (imm18 \times 4)$, a range of about ± 512 KB.

- **Jump targets** are the absolute address $\text{imm26} \times 4$.
- **Large constants.** `lui` sets a register to imm22 shifted left by 10; a following `ori` fills the low 10 bits.

Worked example: `addi r1, r0, 5` is opcode `10`, $a = 1$, $b = 0$, $\text{imm} = 5$, which encodes as `0x40400005`.

Instruction reference

There are 59 instructions. Opcodes are in hexadecimal, and a, b, c are the register fields from the encoding table.

Integer, register form (R)

Op	Mnemonic	Operation	Cycles
01	<code>add a, b, c</code>	$a = b + c$	1
02	<code>sub a, b, c</code>	$a = b - c$	1
03	<code>mul a, b, c</code>	$a = \text{low 32 bits of } b \times c$	4
04	<code>div a, b, c</code>	Signed divide	20
05	<code>divu a, b, c</code>	Unsigned divide	20
06	<code>rem a, b, c</code>	Signed remainder	20
07	<code>remu a, b, c</code>	Unsigned remainder	20
08	<code>and a, b, c</code>	Bitwise and	1
09	<code>or a, b, c</code>	Bitwise or	1
0A	<code>xor a, b, c</code>	Bitwise exclusive or	1
0B	<code>shl a, b, c</code>	Shift left by the low 5 bits of c	1
0C	<code>shr a, b, c</code>	Logical shift right	1
0D	<code>sar a, b, c</code>	Arithmetic shift right	1
0E	<code>slt a, b, c</code>	$a = 1$ if $b < c$ signed, else 0	1
0F	<code>sltu a, b, c</code>	$a = 1$ if $b < c$ unsigned, else 0	1

Integer, immediate form (I, except lui)

Op	Mnemonic	Operation	Cycles
10	addi a, b, imm	$a = b + \text{imm}$, sign-extended	1
11	andi a, b, imm	$a = b \text{ and } \text{imm}$, zero-extended	1
12	ori a, b, imm	$a = b \text{ or } \text{imm}$, zero-extended	1
13	xori a, b, imm	$a = b \text{ exclusive-or } \text{imm}$, zero-extended	1
14	shli a, b, imm	Shift left by the low 5 bits of imm	1
15	shri a, b, imm	Logical shift right	1
16	sari a, b, imm	Arithmetic shift right	1
17	slti a, b, imm	$a = 1$ if $b < \text{imm}$ signed, else 0	1
18	lui a, imm22	$a = \text{imm22}$ shifted left by 10 (U format)	1

Fixed-point and system

Op	Mnemonic	Format	Operation	Cycles
00	brk	—	Halts the cart with an error screen	—
1C	fmul a, b, c	R	$a = (b \times c)$ shifted right 16, using a 64-bit product	4
1D	fdiv a, b, c	R	$a = (b \text{ shifted left } 16) \div c$, using a 64-bit dividend	20
1E	vsync	—	Sleeps until the next frame	Ends the frame

Memory (I)

The address is $b + \text{imm}$, sign-extended. 16-bit and 32-bit accesses must be aligned to their size.

Op	Mnemonic	Operation	Cycles
20	lb a, [b+imm]	Load 8 bits, sign-extend	2
21	lbu a, [b+imm]	Load 8 bits, zero-extend	2
22	lh a, [b+imm]	Load 16 bits, sign-extend	2
23	lhu a, [b+imm]	Load 16 bits, zero-extend	2
24	lw a, [b+imm]	Load 32 bits	2
25	sb a, [b+imm]	Store the low 8 bits of a	2
26	sh a, [b+imm]	Store the low 16 bits of a	2
27	sw a, [b+imm]	Store a	2

Control flow

Branches use the I format and jumps the J format. `jr` and `callr` use the b field only.

Op	Mnemonic	Operation	Cycles
28	beq a, b, label	Branch if a = b	1, or 2 if taken
29	bne a, b, label	Branch if a ≠ b	1, or 2 if taken
2A	blt a, b, label	Branch if a < b, signed	1, or 2 if taken
2B	bge a, b, label	Branch if a ≥ b, signed	1, or 2 if taken
2C	bltu a, b, label	Branch if a < b, unsigned	1, or 2 if taken
2D	bgeu a, b, label	Branch if a ≥ b, unsigned	1, or 2 if taken
2E	jmp label	Jump	2
2F	call label	r15 = pc + 4, then jump	2
30	jr b	Jump to the address in b	2
31	callr b	r15 = pc + 4, then jump to the address in b	2

Vector

Operands written `va`, `vb`, `vc` are vector registers; plain `a`, `b`, `c` are scalar registers. `vld` and `vst` use the I format, `vget` and `vset` use the I format with the lane number (0–3) as the immediate, and the rest use the R format.

Op	Mnemonic	Operation	Cycles
32	<code>vld va, [b+imm]</code>	Load 16 bytes into the four lanes	4
33	<code>vst va, [b+imm]</code>	Store the four lanes	4
34	<code>vmov va, vb</code>	Copy	1
35	<code>vget a, vb, lane</code>	Scalar <code>a</code> = one lane of <code>vb</code>	1
36	<code>vset va, b, lane</code>	One lane of <code>va</code> = scalar <code>b</code>	1
37	<code>vadd va, vb, vc</code>	Lane-wise add	1
38	<code>vsub va, vb, vc</code>	Lane-wise subtract	1
39	<code>vmul va, vb, vc</code>	Lane-wise fixed-point multiply	4
3A	<code>vscale va, vb, c</code>	Every lane multiplied by scalar <code>c</code>	4
3B	<code>vdot a, vb, vc</code>	Scalar <code>a</code> = four-lane dot product	4
3C	<code>vcross va, vb, vc</code>	Cross product of the x, y, z lanes; <code>w</code> = 0	6
3D	<code>vxfm va, vb</code>	<code>va</code> = matrix × <code>vb</code> , with the matrix rows in <code>v4</code> – <code>v7</code>	8
3E	<code>vproj va, vb</code>	Perspective divide to screen coordinates	12

vdot sums the four 64-bit lane products, then shifts right by 16. For a three-component dot product, keep the `w` lanes at zero.

vxfm computes each output lane as the dot product of one matrix row with `vb`: `x` from `v4`, `y` from `v5`, `z` from `v6`, `w` from `v7`. The result is computed in full before it is written, so `va` may be one of the matrix registers.

vproj maps a transformed vertex to the 320×240 screen:

- $x = 160 + (160 \times x \div w)$, rounded down to a whole pixel
- $y = 120 - (120 \times y \div w)$, rounded down to a whole pixel
- $z = z \div w$, as 16.16

- `w` is unchanged

The `x` and `y` results are plain integers, not 16.16, and are clamped to $-1024..1023$. The projection matrix carries the field of view and aspect ratio, so there are no separate projection registers.

Reserved opcodes

Opcodes `19` – `1B`, `1F` and `3F` fault. Because `00` is `brk`, both the all-zeros word and the all-ones word stop the cart, which catches execution running into empty memory.

Assembler shorthands

Shorthand	Expands to
<code>nop</code>	<code>add r0, r0, r0</code>
<code>mov a, b</code>	<code>add a, b, r0</code>
<code>ret</code>	<code>jr r15</code>
<code>li a, const</code>	One <code>addi</code> if the constant fits 18 signed bits, otherwise <code>lui</code> then <code>ori</code>

Timing and faults

The CPU gets 500,000 cycles per frame, about 30 MHz at 60 fps, and every instruction has a fixed cost. This is the console's real CPU limit.

Frame loop. A cart does its work for the frame, then executes `vsync`. The framebuffer swap, input is sampled, and the cycle counter resets to 500,000.

Overrun. If the budget runs out before `vsync`, the display repeats the previous frame and the CPU continues with a fresh budget. A cart that needs between one and two budgets per frame therefore runs at 30 fps, as slowdown did on real hardware.

What the budget buys. A vertex transform (`vld`, `vxfm`, `vproj`, `vst`) costs 28 cycles. A worst-case scene of 2,000 triangles with no shared vertices spends about 170,000 cycles on transforms and roughly 80,000 on building packets. That is half the frame, leaving the other half for game logic.

Faults. A fault halts the cart and shows an error screen with the fault type and the address of the instruction.

Fault	Cause
Break	A <code>brk</code> instruction, including the all-zeros word
Illegal instruction	A reserved opcode, non-zero unused bits in an R-format word, or a vector register field above 7
Misaligned access	A 16-bit or 32-bit access, or a <code>vld / vst</code> , at an address that is not a multiple of its size (4 for vectors)
Unmapped address	A read, write or jump outside the memory map, or to an unused I/O offset
Read-only write	A write to cart ROM or to a read-only I/O register
Bad I/O width	An 8-bit or 16-bit access to an I/O register

Divide rules. These do not fault.

- Dividing by zero gives 0, for `div`, `divu`, `rem`, `remu`, `fdiv`, and inside `vproj`.
- The most negative number divided by `-1` returns itself, with remainder 0.
- An `fdiv` result too large for 32 bits keeps its low 32 bits.

Memory map and VRAM layout

All addresses fit in 24 bits. Anything outside the four regions below is unmapped and faults.

Address	Region	Size	Access
<code>0x000000 - 0x1FFFFFF</code>	RAM	2 MB	Read/write
<code>0x200000 - 0x3FFFFFF</code>	Cart ROM	2 MB	Read only
<code>0x400000 - 0x4FFFFFF</code>	VRAM	1 MB	Read/write
<code>0xFF0000 - 0xFF03FF</code>	I/O registers	1 KB	Per register

VRAM is ordinary memory to the CPU, so carts can write pixels, palettes and textures directly.

Address	Contents	Size
0x400000 – 0x4257FF	Framebuffer A	153,600 bytes
0x425800 – 0x44AFFF	Framebuffer B	153,600 bytes
0x44B000 – 0x44BFFF	Spare	4 KB
0x44C000 – 0x44DFFF	Palette memory, 4,096 colours	8 KB
0x44E000 – 0x47FFFF	Spare	200 KB
0x480000 – 0x4FFFFFF	16 texture slots	32 KB each

Framebuffer. The pixel at (x, y) is the 16-bit value at $\text{base} + (y \times 320 + x) \times 2$. Which buffer is the back buffer changes each frame; read `GPU_BACK` to find it.

Palettes. Colour i is the 16-bit value at `0x44C000` + $i \times 2$, in the same format as a framebuffer pixel. The 4,096 colours are viewed two ways:

- An 8-bit texture picks one of 16 palettes of 256 colours; palette n starts at colour $n \times 256$.
- A 4-bit texture picks one of 256 palettes of 16 colours; palette n starts at colour $n \times 16$.

Textures. Every texture is 256×256 texels, and slot n starts at `0x480000` + $n \times 0x8000$.

Depth	Storage	Texel (u, v) is at	Slots used	Fits in VRAM
4-bit	Two texels per byte; the low half is the left texel	Byte $v \times 128 + u \div 2$	1	16 textures
8-bit	One texel per byte	Byte $v \times 256 + u$	2, starting on an even slot	8 textures

Loading. Carts copy textures from ROM into VRAM with ordinary loads and stores. Filling all 16 slots costs roughly 900,000 cycles, about two frames.

I/O registers

Every register is 32 bits wide and accessed with `lw` and `sw`. `lui r, 0x3FC0` puts the base address `0xFF0000` in a register, and every I/O register is then a small offset from it.

GPU (`0xFF0000`)

Offset	Name	Access	Purpose
+00	GPU_DRAW	Write	Address of the first packet of a list; writing it draws the list into the back buffer
+04	GPU_CLEAR	Write	Fills the back buffer with a 15-bit colour
+08	GPU_CTRL	Read/write	Bit 0: dither on
+0C	GPU_STATUS	Read	Bits 0–15: triangles drawn this frame. Bit 16: limit exceeded, extras dropped
+10	GPU_BACK	Read	VRAM address of the current back buffer

A cart may write `GPU_DRAW` several times in a frame, for example once for the world and once for the interface. The triangle count resets at `vsync`.

Audio (`0xFF0100`)

There are eight channels of 32 bytes each. Channel `n` starts at `0xFF0100 + n × 0x20`.

Offset	Name	Access	Purpose
+00	ADDR	Read/write	Address of the sample data, in RAM or ROM
+04	LEN	Read/write	Length in samples
+08	LOOP	Read/write	Sample index to return to when looping
+0C	PITCH	Read/write	Playback rate as 16.16; 1.0 plays at 22,050 Hz
+10	VOL	Read/write	Bits 0–7: left volume. Bits 8–15: right volume
+14	CTRL	Read/write	Bit 0: start playing from the beginning. Bit 1: loop. Bit 2: 16-bit samples (otherwise 8-bit)
+18	POS	Read	Current sample index

Offset	Name	Access	Purpose
+1C	—	—	Reserved

Samples are signed, mono and uncompressed. One second costs about 22 KB at 8 bits and 44 KB at 16 bits.

Input (0xFF0200)

Offset	Name	Access	Purpose
+00	PAD1	Read	Buttons held on controller 1, one bit each
+04	PAD2	Read	Buttons held on controller 2
+08	STICK1_X	Read	Controller 1 stick: -1.0 (left) to 1.0 (right), as 16.16
+0C	STICK1_Y	Read	Controller 1 stick: -1.0 (down) to 1.0 (up), as 16.16
+10	STICK2_X	Read	Controller 2 stick, horizontal
+14	STICK2_Y	Read	Controller 2 stick, vertical

Bit	0	1	2	3	4	5	6	7	8	9	10
Button	Up	Down	Left	Right	A	B	X	Y	L	R	Start

- **Sampling.** Buttons and sticks are read once per frame at `vsync` , which keeps replays deterministic.
- **Just pressed.** Carts detect a new press by comparing with the previous frame's value.
- **Dead zone.** The console applies it, so a resting stick reads exactly 0.
- **Keyboard.** Keys map the stick to full deflection in eight directions.

System (0xFF0300)

Offset	Name	Access	Purpose
+00	FRAME	Read	Frames since reset
+04	CYCLES	Read	Cycles left in this frame

Offset	Name	Access	Purpose
+08	RAND	Read/write	Read: next pseudo-random number. Write: set the seed
+0C	DEBUG	Write	Prints the low 8 bits as one character to the debug console

Access rules

- Only 32-bit access is allowed; byte or halfword access faults.
- Reading a write-only register returns 0.
- Writing a read-only register faults, as does any access to an offset not listed here.

GPU packets

The GPU understands one thing: a linked list of packets in memory, which it draws in list order. Sorting is the cart's job, done by choosing where in the list each packet goes.

Header

Every packet starts with one header word.

Bits	Field
0-23	Address of the next packet, or 0xFFFFFFFF for the end of the list
24-31	Packet type

Type 00 is an empty packet that only links onward. Types the GPU does not recognise are skipped. A next address outside RAM and ROM faults, as does a list longer than 65,536 packets.

The ordering table

The ordering table is a convention built from empty packets, not something the GPU knows about. It gives back-to-front drawing without sorting.

1. **Reset.** Each frame, the cart fills an array of 1,024 empty packets. Entry i links to entry $i - 1$, and entry 0 ends the list. This costs about 4,000 cycles.
2. **Insert.** For each polygon, the cart picks a bucket from its depth, 0 for nearest and 1,023 for farthest. It copies the bucket entry's next address into the new packet, then points the bucket entry at the new packet.
3. **Draw.** The cart writes the address of entry 1,023 to `GPU_DRAW`. The GPU walks from the farthest bucket to the nearest.

Within one bucket, the most recently inserted packet draws first. Polygons at similar depths can therefore draw in the wrong order, which is the source of the sorting glitches.

Polygon types

A polygon's type is `20` plus four flags, giving `20 - 2F`.

Bit	Meaning when set
0	Gouraud: one colour per vertex (otherwise one colour for the whole polygon)
1	Textured
2	Quad: four vertices (otherwise three)
3	Semi-transparent

Words

After the header, each vertex contributes its words in this order:

1. **Colour:** for every vertex if Gouraud, for the first vertex only if flat.
2. **Position:** always.
3. **Texture coordinate:** only if textured.

Word	Bits	Field
Colour	0-7, 8-15, 16-23	Red, green, blue
Colour (first only)	24-25	Blend mode, used when the packet is semi-transparent

Word	Bits	Field
Position	0–15	x, signed whole pixels
Position	16–31	y, signed whole pixels
Texture coordinate	0–7, 8–15	u, v
Texture coordinate (first only)	16–19	Texture slot, 0–15
Texture coordinate (first only)	20	Set for a 4-bit texture, clear for 8-bit
Texture coordinate (first only)	24–31	Palette: 0–255 for 4-bit, 0–15 for 8-bit

All other bits should be zero.

Shape	Flat	Gouraud	Flat textured	Gouraud textured
Triangle	20 bytes	28 bytes	32 bytes	40 bytes
Quad	24 bytes	36 bytes	40 bytes	52 bytes

For example, a Gouraud textured triangle (type 23) is ten words: header, then colour, position and texture coordinate for each of three vertices. A worst-case frame of 2,000 of these uses 80 KB of RAM.

Blend modes

Blending works on the 5-bit framebuffer channels after tinting and dither, and each channel clamps to 0–31.

Mode	Result	Typical use
0	Half background + half polygon	Glass, water, ghosts
1	Background + polygon	Fire, glows, light beams
2	Background – polygon	Shadows, dark smoke
3	Background + a quarter of the polygon	Faint glows

Transparent polygons sort through the ordering table like everything else, so they blend over whatever was drawn before them.

Drawing rules

- **Quads** are drawn as two triangles, vertices 0-1-2 and 1-2-3, and count as two against the 2,000 limit.
- **Palette index 0** is skipped in textured polygons, in both texture depths, whether or not the packet is semi-transparent.
- **Clipping** is to the screen edges only. Polygons that cross behind the camera must be dropped or clipped by the cart before projection.
- **Culling** of back faces is the cart's job, which also saves building packets for hidden faces.
- **2D drawing** (interface, text, sprites) uses textured quads; there is no separate sprite packet.

Fog

The GPU has no fog. Carts blend each vertex colour toward the fog colour before submitting, which costs CPU cycles per vertex. Untextured polygons can fog to any colour. Textured polygons can only fade toward dark colours, because tinting multiplies the texel. An additive layer drawn over distant geometry is a partial workaround for bright fog.

Language

Carts are written in a custom, statically typed language that compiles to the CPU's machine code. The type names are settled; the syntax is not.

Type	Meaning	Size	Lives in
s8 s16 s32	Signed integers	1, 2, 4 bytes	Scalar register
u8 u16 u32	Unsigned integers	1, 2, 4 bytes	Scalar register
fixed	16.16 fixed-point number	4 bytes	Scalar register
bool	True or false	1 byte	Scalar register
vec2 vec3 vec4	Vectors of fixed	16 bytes	Vector register

Type	Meaning	Size	Lives in
<code>ivec4</code>	Four <code>s32</code> lanes	16 bytes	Vector register
<code>mat4</code>	4×4 matrix of <code>fixed</code>	64 bytes	Four vector registers

The 8- and 16-bit integers exist for memory layout: pixels, texels, packets and sample data. Arithmetic happens at 32 bits. Proposed: every vector type occupies 16 bytes in memory with unused lanes zero, so `vld` and `vst` work on all of them directly.

Features.

- **No garbage collector.** Data is structs and fixed-size arrays in the 2 MB of RAM, so there are no pauses and the memory limit is real.
- **Fixed-point literals.** A literal such as `0.05` is a `fixed` constant; there is no floating point.
- **Vector operators.** Arithmetic on vector types compiles to vector instructions.
- **Named registers.** A memory-mapped register can be declared with a name and address, so an assignment to it compiles to a single store.
- **Standard library.** Calls such as `camera`, `mesh` and `btn` hide the ordering table, packet building, fog and input edge detection. Carts can bypass them and build packets by hand.

An illustrative sketch, not final syntax:

```
struct Player { pos: vec3, yaw: fixed }

var p: Player

fn update() {
    if btn(LEFT) { p.yaw -= 0.05 }
    if btn(RIGHT) { p.yaw += 0.05 }
}

fn draw() {
    camera(p.pos, p.yaw)
    mesh(LEVEL)
}
```

Open questions

Nine things are undecided, and eight details in this draft were filled in without discussion and need a check.

Not yet decided

- ☐ **Name.** "Mei" is a working name.
- ☐ **Save data.** Nothing persists between sessions. Saving progress needs a small save-memory region.
- ☐ **Cart file format.** How code, assets and metadata are packed into the 2 MB ROM image.
- ☐ **Language syntax.** Only the type names are settled.
- ☐ **Compressed audio.** Uncompressed samples are costly against a 2 MB cart; a compressed format needs a decoder.
- ☐ **Standard library location.** Compiled into each cart, counting against its 2 MB, or held in a built-in ROM.
- ☐ **Culling and clipping helpers.** A back-face test instruction would make culling cheaper than scalar code.
- ☐ **Fill rate.** Only the triangle count is limited, so large overlapping polygons are free.
- ☐ **Controller count.** Two are assumed.

Filled in while drafting

- **Pixel layout.** Red in bits 0–4, green in 5–9, blue in 10–14, matching the PlayStation.
- **Dither.** The matrix values, and that dither applies to every polygon when enabled.
- **Random numbers.** The algorithm must be fixed for determinism; none is specified yet.
- **Vector storage.** All vector types take 16 bytes in memory.
- **Overrun.** A missed `vsync` repeats the previous frame.
- **List safety.** Unknown packet types are skipped, and a list is capped at 65,536 packets.
- **Edge cases.** An `fdiv` overflow keeps its low 32 bits, and unused bits in R-format words must be zero.
- **Audio mixing.** How the eight channels are summed and clipped is unspecified.

Implementation notes

Use SDL3 as the platform layer (window, presenting the framebuffer, audio, controller input). The emulator rasterizes triangles in its own C code.

What SDL3 is good for

Job	SDL3 feature
Showing the framebuffer	One 320×240 streaming texture in a 15-bit format matching VRAM, updated each frame, scaled with nearest-neighbour at whole-number multiples
Audio	An audio stream fed with the mixed samples at 22,050 Hz
Input	The gamepad interface for d-pad, buttons and stick, plus keyboard events
Main loop	The callback-style entry point, which suits browsers
Browser build	Emscripten carries an SDL3 port (discussion)

The exact function names are from memory and should be checked against the SDL3 wiki when work starts.

Trade-off. SDL3 adds to the WebAssembly download. The alternative in the browser is a small hand-written JavaScript layer using canvas, Web Audio and the Gamepad API. Keep the platform layer behind three calls (present a frame, queue audio, read input) so either can be used. Start with SDL3 on desktop, where debugging is fastest.

Structure

The core is plain C with no platform calls. Given a cart, it runs one frame and produces a 320×240 16-bit image and about 368 audio samples.

Module	Responsibility
Bus	Memory map, I/O register dispatch, fault checks
CPU	Instruction decode and execution, cycle counting

Module	Responsibility
GPU	Packet list walking and triangle rasterization into VRAM
Audio	Mixing eight sample channels
Platform	SDL3 or JavaScript: display, sound output, input
Assembler	Text to machine code
Compiler	The language to machine code

C pitfalls

- **Signed overflow** is undefined in C. Do `add`, `sub` and `mul` on unsigned 32-bit values.
- **Arithmetic right shift** of a negative value is implementation-defined. Implement `sar` explicitly.
- **Division** must check for zero and for the most negative number divided by `-1` before dividing.
- **Rounding down.** C division rounds toward zero, but `vproj` rounds toward negative infinity.
- **Wide intermediates.** `fmul`, `fdiv`, `vmul`, `vdot` and `vxfm` need 64-bit intermediates.
- **Fill rule.** Use integer edge functions with a top-left rule so pixels on a shared edge are drawn once. Otherwise semi-transparent meshes blend twice along their seams.

Build order

1. Bus and CPU interpreter, with a test per instruction.
2. Assembler, so test programs can be written by hand.
3. GPU rasterizer: flat triangles first, then Gouraud, textures, dither and blending.
4. SDL3 platform layer, to see pixels and press buttons.
5. Audio.
6. Compiler and standard library.
7. WebAssembly build.